**The Chinese University of Hong Kong**

**Faculty of Mathematics**




# Formal Verification and Interactive Theorem Proving in Lean 4

*Individual Research Report & Reference Manual*


**Author:** Chan Kwan Ho (1155194495)

**Supervisor:** Prof. Mykola Matviichuk


**Research Domain:** Interactive Theorem Proving & Type Theory

(Lean 4 Formalization Research)

November 2, 2026

# Declaration

I hereby declare that this research report titled **"The Elliptic Log Bundle: Theory and Formalization in Lean 4"** is my own original work, prepared for my personal reference and academic evaluation.

Where this work builds upon collaborative discussions or shared research activities conducted within a team setting with my teammate Garry Gao ShiQing, all literature synthesis, written text, and the Lean 4 formalization of the elliptic log bundle presented in **Section 3** have been composed solely by me, with help of AI under supervision of Prof. Mykola Matviichuk, except where explicitly attributed to external literature or the Lean mathematical library (`Mathlib4`).

**Author Signature:**

---

Your Full Name
Date: November 2, 2026

# Abstract

Log symplectic Poisson structures on projective varieties are governed by combinatorial invariants encoded in biresidue matrices. Building upon the classification of elliptic log symplectic brackets established in Section 3 of Matviichuk (2023), this report presents a fully machine-checked **Lean 4** library formalizing the structural theory of biresidue matrices containing smoothable cycles.

Utilizing `Mathlib4`'s linear algebra and graph theory infrastructure, we construct type-theoretic encodings and tactic-driven proofs for three foundational results:

(i) the cyclic degree bounds for smoothable cycles (**Lemma 3.1**) ;

(ii) the complete four-family classification of full smoothable cycles (**Theorem C**); and

(iii) the spectral rank computation executed via root-of-unity analysis (**Lemma 3.3**).

This work demonstrates how interactive proof assistants can rigorously verify classification theorems, combinatorial structures, and spectral invariants in arithmetic and Poisson geometry.

# Acknowledgements

I wish to express my sincere gratitude to my supervisor for their insightful guidance and feedback throughout the preparation of this report.

I am also thankful to my project collaborator Gao Shiqing for engaging discussions, as well as the open-source Lean community and maintainers of `Mathlib4` for providing the interactive theorem proving infrastructure that made the Lean formalization of the elliptic log bundle in Section 3 possible.

# Contents

**Declaration** i

**Abstract** ii

**Acknowledgements** iii

**1 Introduction** 1
- 1.1 Overview and Motivation . . . 1
- 1.2 Structure of the Report . . . 2

**2 Theoretical Foundations** 3
- 2.1 Introduction . . . 3
- 2.2 Basic Result . . . 3

**3 Lean 4 Verification** 8
- 3.1 Construction of Definition . . . 8
- 3.2 Lemma 2.1 . . . 8
  - 3.2.1 Skew-Symmetric . . . 8
  - 3.2.2 Smoothable Edge & Lemma 2.1.2 . . . 9
  - 3.2.3 Prove of Lemma 2.1.1 . . . 10
- 3.3 the Families . . . 10
  - 3.3.1 Family $\mathcal{Y}$ . . . 10
  - 3.3.2 Family $\mathcal{Z}$ . . . 13
- 3.4 Errors and Modifications . . . 14
- 3.5 Theorem C . . . 15
  - 3.5.1 Row sum zero - AddSumLemma.lean . . . 15
  - 3.5.2 Skew-Symm - . . . 15
  - 3.5.3 edge $k - k + 1 \pmod{n}$ is colored - . . . 16
- 3.6 Mathematical Verification Process and Frequently Missed Preconditions 16

**4 Formalization Insights & Technical Challenges** 18
- 4.1 Finset.sum vs Fin sum . . . 18
- 4.2 Family $Y$ . . . 18
- 4.3 zRowk_sum_zero . . . 22

**5 Conclusion & Future Work** **25**

# List of Figures

3.1 picture of `# eval Y 1` . . . . . 12
3.2 picture of helper function . . . . . 12
3.3 `# eval IO.println (repr (Y 1))` . . . . . 13
3.4 picture of `#eval IO.println (repr (Z 1))` . . . . . 14

# List of Tables

4.1 Formalization Challenges, Observed Tactics, and Structural Insights . 21
4.2 Key Reminders for Lean 4 Sequence Sum Formalization . . . . . 23

# Listings

# Chapter 1

# Introduction

## 1.1 Overview and Motivation

Until recently—roughly two years ago—formal verification was widely regarded as a specialized, niche field within computer science. Because artificial intelligence had not yet demonstrated advanced reasoning capabilities, the broader mathematical and scientific communities paid relatively little attention to machine-checked proofs, viewing them as excessively tedious and disconnected from practical research.

However, over the past year, the explosive advancement of AI models has fundamentally altered this perspective. As AI systems are increasingly tasked with generating complex, multi-step reasoning, a critical question has emerged: *How can we verify that an AI model is reasoning logically and producing strictly correct conclusions?* Pure mathematics provides the ultimate, unambiguous benchmark to test these logical capabilities.

This intersection of AI and logic reached a historic milestone when leading AI labs (such as OpenAI and Google DeepMind) began leveraging the **Lean 4** interactive theorem prover to formally verify high-level mathematics. The ongoing, ambitious efforts to formalize landmark achievements like Fermat's Last Theorem in Lean sent shockwaves through the community. Traditionally, as noted in personal communications with Professor Mykola Matviichuk, the manual formalization of advanced mathematics was an extraordinarily slow and grueling process, requiring immense time and effort from expert mathematicians, including dedicated research groups at Imperial College London. The realization that AI could potentially accelerate or assist in this painstaking formalization process marked a major turning point.

Motivated by this sudden convergence of rapid AI development, logical safety, and interactive theorem proving, this technical report bridges pure mathematics and formal logic. By building a machine-checked Lean 4 library for the classification of biresidue matrices and smoothable cycles [**matviichuk2023**], this project contributes to the growing effort of digitizing advanced geometric structures and exploring how complex modern research can be rigorously verified by computers.

## 1.2 Structure of the Report

The remainder of this report is organized into five chapters as follows:

- **Chapter 1: Introduction** – Outlines the research context, contrasting classical manual formalization with the recent surge in AI-driven formal verification. It establishes the core motivations for machine-checking advanced mathematics and summarizes the primary contributions of this project.
- **Chapter 2: Theoretical Foundations** – Provides the necessary mathematical background on elliptic curves, logarithmic structures, and vector bundles. It presents the pen-and-paper definitions, supporting lemmas, and main theorems that establish the geometric framework of the elliptic log bundle.
- **Chapter 3: Lean 4 Formalization** – Details the primary technical deliverable of this work: mechanizing the elliptic log bundle using the **Lean 4** interactive theorem prover and its mathematical library, `Mathlib4`. It introduces the core data structures, type-theoretic encodings, and tactic-driven proofs.
- **Chapter 4: Formalization Challenges & Machine Logic** – Investigates the technical friction between human mathematical intuition and machine-checked formal logic. It discusses edge cases, type-class resolution bottlenecks, and proof engineering strategies developed during the formalization process.
- **Chapter 5: Conclusion and Outlook** – Summarizes the main results of the project, outlines potential avenues for future research, and reflects on the broader impact of formal verification tools on the future of the mathematical research community.

# Chapter 2

# Theoretical Foundations

## 2.1 Introduction

In Section 2, the author establishes that a semi-toric log symplectic manifold (defined on $\mathbb{P}^{n-1} \times \mathbb{D}^m$) has polar/degeneracy divisors corresponding to coordinate hyperplanes $D_i = \{y_i = 0\}$.

The geometry of these hyperplanes is controlled by an $n \times n$ skew-symmetric biresidue matrix $B = (b_{ij})$.

When an edge $i \to j$ satisfies a specific arithmetic condition ($\theta_{ijk} = \frac{b_{jk}+b_{ki}}{b_{ij}} \in \mathbb{Z}_{\geq 0}$), it is called smoothable. A smoothable edge allows a Poisson deformation $\rho_{ij}$ that smoothes out the singular intersection $D_i \cap D_j$. Theorem 2.5 proves that multiple smoothable edges can be deformed jointly without obstruction.

The author's core hypothesis is that whenever the smoothable edges form a closed cycle containing all $n$ vertices (a "full smoothable cycle"), the resulting multi-parameter Poisson deformation is guaranteed to be elliptic.

## 2.2 Basic Result

**Goal:** This section addresses the purely combinatorial problem: Which $n \times n$ biresidue matrices $B$ with zero row sums contain a full cycle of smoothable edges?

**Definition 2.1** (Biresidue Matrix)**.** A matrix $B = (b_{ij})_{i,j=0}^{n-1} \in M_{n\times n}(\mathbb{C})$ is called a *biresidue matrix* if it is skew-symmetric, i.e., $b_{ij} = -b_{ji}$ for all $0 \leq i, j \leq n-1$ (which implies $b_{ii} = 0$).

We say that $B$ has *zero row sums* if for every $i \in \{0, \ldots, n-1\}$,

$$\sum_{j=0}^{n-1} b_{ij} = 0.$$

**Definition 2.2** (Smoothability Quotient and Smoothable Edge)**.** Let $B = (b_{ij})_{i,j=0}^{n-1}$ be a biresidue matrix.

1. For distinct indices $i, j \in \{0, \ldots, n-1\}$ with $b_{ij} \neq 0$ and any $k \notin \{i, j\}$, define the *biresidue quotient*
$$\theta_{ijk} := \frac{b_{jk} + b_{ki}}{b_{ij}}.$$
Note that skew-symmetry implies symmetry in $i$ and $j$: $\theta_{ijk} = \theta_{jik}$.

2. An undirected edge $\{i, j\}$ (denoted $i - j$) is called *smoothable* if:
   (a) $b_{ij} \neq 0$, and
   (b) $\theta_{ijk} \in \mathbb{Z}_{\geq 0}$ for all $k \in \{0, \ldots, n-1\} \setminus \{i, j\}$.

**Lemma 2.1** (Properties of Smoothable Edges)**.** *Let $B = (b_{ij})_{i,j=0}^{n-1}$ be a biresidue matrix with zero row sums.*

1. *For each vertex $j \in \{0, \ldots, n-1\}$, there are at most two smoothable edges incident to $j$.*
2. *If $i - j$ and $j - k$ are distinct smoothable edges incident to $j$, then $\frac{b_{ji}}{b_{jk}} \in \mathbb{Q}_{<0}$.*

*Proof.* We first show 2: Since $i - j$ and $j - k$ are distinct smoothable edges, then
$$\frac{b_{jk} + b_{ki}}{b_{ij}} = p \in \mathbb{Z}_{\geq 0} \quad , \quad \frac{b_{ki} + b_{ij}}{b_{jk}} = q \in \mathbb{Z}_{\geq 0}$$
Where we substitute $k, i$ into the entries part correspondingly. Then, we obtained the two equality: $b_{jk} + b_{ki} = p \cdot b_{ij}$ and $b_{ki} + b_{ij} = q \cdot b_{jk}$ and that $b_{jk} - b_{ij} = p \cdot b_{ij} - q \cdot b_{jk}$, we obtain $(1 + q)b_{jk} = (1 + p)b_{ij}$.

To show 1, suppose that there exists $i - j_1, i - j_2, i - j_3$ are distinct smoothable edges. Then, $\frac{b_{ij_1}}{b_{ij_2}} = a \in \mathbb{Q}_{<0}$, $\frac{b_{ij_2}}{b_{ij_3}} = b \in \mathbb{Q}_{<0}$. Note that by $\frac{b_{ij_1}}{b_{ij_3}} = \frac{b_{ij_1}}{b_{ij_2}} \cdot \frac{b_{ij_2}}{b_{ij_3}} = a \cdot b \in \mathbb{Q}_{>0}$, contradicting that fact that $\frac{b_{ij_1}}{b_{ij_3}} \in \mathbb{Q}_{<0}$. □

*Remark.* Since $b_{ij} = -b_{ji}$, the relation $\frac{b_{ji}}{b_{jk}} \in \mathbb{Q}_{<0}$ implies that $\frac{b_{ij}}{b_{jk}} \in \mathbb{Q}_{>0}$. Thus, $b_{ij}$ is a positive rational multiple of $b_{jk}$.

**Proposition 2.2** (Graph Topology of Smoothable Edges)**.** *Let $G_B = (V, E_s)$ be the graph with vertex set $V = \{0, 1, \ldots, n-1\}$ and edge set $E_s = \{\{i, j\} \mid i - j$ is smoothable$\}$. Since the degree of every vertex in $G_B$ is at most 2 (by Lemma 2.3(1)), every connected component of $G_B$ is either a simple path (chain) or a simple cycle.*

**Definition 2.3** (Smoothable Cycle and Full Smoothable Cycle)**.** A connected component of $G_B$ that forms a simple cycle is called a *smoothable cycle*.

If $G_B$ consists of a single smoothable cycle containing all $n$ vertices $\{0, 1, \ldots, n-1\}$, we say $B$ has a *full smoothable cycle*.

Before dive into the technical part, i would like to introduce the main theorem Theorem C, and the corresponding Family Type: $\mathcal{C}_{n,k}, \mathcal{C}_{n,k,I}, \mathcal{X}_4\mathcal{X}_5, \mathcal{Y}_{2(2k+1)}, \mathcal{Z}_{5(2k+1)}$.

**Definition 2.4** (Cyclic shift operator)**.** For row vector $\mathbf{r} = (r_0, ..., r_{n-1})$, we denote $T\mathbf{r}$, or $T(\mathbf{r})$, the cyclic shift $(r_{n-1}, r_0, ..., r_{n-2})$ of $\mathbf{r}$ by one position to the right.

**Definition 2.5** (**Family** $\mathcal{C}_{n,k}$)**.** For any numbers $n \geq 3$, and $1 \leq k \leq \frac{n}{2}$, we define the matrix $\mathcal{C}_{n,k}$ whose zeroth row is

$$\mathbf{r}(k) := (0 \underbrace{1 \cdots 1}_{k} \underbrace{0 \cdots 0}_{n-1-2k} \underbrace{-1 \cdots -1}_{k})$$

and whose $i-$th row is $T^i\mathbf{r}(k)$.

**Example 2.1.**

$$\mathcal{C}_{5,1} = \begin{pmatrix} 0 & 1 & 0 & 0 & -1 \\ -1 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 1 \\ 1 & 0 & 0 & -1 & 0 \end{pmatrix}, \mathcal{C}_{7,3} = \begin{pmatrix} 0 & 1 & 1 & 1 & -1 & -1 & -1 \\ -1 & 0 & 1 & 1 & 1 & -1 & -1 \\ -1 & -1 & 0 & 1 & 1 & 1 & -1 \\ -1 & -1 & -1 & 0 & 1 & 1 & 1 \\ 1 & -1 & -1 & -1 & 0 & 1 & 1 \\ 1 & 1 & -1 & -1 & -1 & 0 & 1 \\ 1 & 1 & 1 & -1 & -1 & -1 & 0 \end{pmatrix}$$

**Definition 2.6** (**Family** $\mathcal{C}_{n,k,I}$)**.** For $n \geq 3, 1 \leq k < \frac{n}{2}$ and $I = \{i_0, i_1, ..., i_{d-1}\}$ being a list of $0's, 1's$ of the length $d = \gcd(n, k)$, we define matrix $\mathcal{C}_{n,k,I}$ as follows:

- $\mathrm{r}_0 = \mathrm{r}(k - i_0), \mathrm{r}_1 = T(\mathrm{r}(k - i_1)), ..., \mathrm{r}_{d-1} = T^{d-1}(\mathrm{r}(k - i_{d-1}))$.
- For $d \leq i \leq n - 1$, the i-th row $\mathrm{r}_i$ is defined by the d-periodicity condition : $\mathrm{r}_i = T^d\mathrm{r}_{i-d}$.

**Example 2.2.** For example, $\mathcal{C}_{n,k,\{0,0,...,0\}} = \mathcal{C}_{n,k}; \mathcal{C}_{9,3,\{1,1,...,1\}} = \mathcal{C}_{n,k-1}$;

$$\mathcal{C}_{9,3,\{0,1,1\}} = \begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 0 & -1 & -1 & -1 \\ -1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & -1 \\ -1 & -1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ -1 & -1 & -1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & -1 & -1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & -1 & -1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & -1 & -1 & -1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & -1 & -1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 & 0 & -1 & -1 & 0 \end{pmatrix} = \begin{pmatrix} \mathrm{r}(3) \\ T(\mathrm{r}(2)) \\ T^2(\mathrm{r}(2)) \\ T^3(\mathrm{r}(3)) \\ T^4(\mathrm{r}(2)) \\ T^5(\mathrm{r}(2)) \\ T^6(\mathrm{r}(3)) \\ T^7(\mathrm{r}(2)) \\ T^8(\mathrm{r}(2)) \end{pmatrix}$$

**Definition 2.7** (**Family** $\mathcal{Y}_{2(2k+1)}$)**.** For $k \geq 1$, the first two rows $\mathrm{r}_0, \mathrm{r}_1$ of $\mathcal{Y}_{2(2k+1)}$ are defined follows:

- $\begin{pmatrix} 0 & 2 & 1 & 1 & \cdots & 1 & -1 & -1 & -1 & \cdots & -1 \\ -2 & 0 & 1 & 1 & \cdots & 1 & 1 & 1 & -1 & \cdots & -1 \end{pmatrix}$
- where zeroth row has $\frac{n}{2} - 2$ ones, for $n = 2(2k+1)$.
- For $2 \leq i \leq n-1$, the i-th row of the matrix $\mathcal{Y}_{2(2k+1)}$ is determined by the following periodicity conditions: $\mathrm{r}_i = T^2(r_{i-2})$.
- For instance, the $\mathrm{r}_2$ is the cyclic shift of $\mathrm{r}_0$ to the right 2 times.

**Example 2.3.** $\mathcal{Y}_6 = \begin{pmatrix} 0 & 2 & 1 & -1 & -1 & -1 \\ -2 & 0 & 1 & 1 & 1 & -1 \\ -1 & -1 & 0 & 2 & 1 & -1 \\ 1 & -1 & -2 & 0 & 1 & 1 \\ 1 & -1 & -1 & -1 & 0 & 2 \\ 1 & 1 & 1 & -1 & -2 & 0 \end{pmatrix}$,

$$\mathcal{Y}_{10} = \begin{pmatrix} 0 & 2 & 1 & 1 & 1 & -1 & -1 & -1 & -1 & -1 \\ -2 & 0 & 1 & 1 & 1 & 1 & 1 & -1 & -1 & -1 \\ -1 & -1 & 0 & 2 & 1 & 1 & 1 & -1 & -1 & -1 \\ -1 & -1 & -2 & 0 & 1 & 1 & 1 & 1 & 1 & -1 \\ -1 & -1 & -1 & -1 & 0 & 2 & 1 & 1 & 1 & -1 \\ 1 & -1 & -1 & -1 & -2 & 0 & 1 & 1 & 1 & 1 \\ 1 & -1 & -1 & -1 & -1 & -1 & 0 & 2 & 1 & 1 \\ 1 & 1 & 1 & -1 & -1 & -1 & -2 & 0 & 1 & 1 \\ 1 & 1 & 1 & -1 & -1 & -1 & -1 & -1 & 0 & 2 \\ 1 & 1 & 1 & 1 & 1 & -1 & -1 & -1 & -2 & 0 \end{pmatrix}$$

**Definition 2.8** (**Family $\mathcal{Z}_{5(2k+1)}$**)**.** For $k \geq 1$, the first five rows $\mathrm{r}_0, \mathrm{r}_1$ of $\mathcal{Y}_{2(2k+1)}$ are defined follows:

- $$\begin{pmatrix} \mathrm{r}_0 \\ \mathrm{r}_1 \\ \mathrm{r}_2 \\ \mathrm{r}_3 \\ \mathrm{r}_4 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 1 & \cdots & 1 & -1 & -1 & -1 & -1 & -1 & -1 & \cdots & -1 \\ -1 & 0 & 2 & 2 & 1 & 1 & \cdots & 1 & -1 & -1 & -1 & -1 & -1 & -1 & \cdots & -1 \\ -1 & -2 & 0 & 2 & 1 & 1 & \cdots & 1 & 1 & 1 & -1 & -1 & -1 & -1 & \cdots & -1 \\ -1 & -2 & -2 & 0 & 1 & 1 & \cdots & 1 & 1 & 1 & 1 & 1 & -1 & -1 & \cdots & -1 \\ -1 & -1 & -1 & -1 & 0 & 1 & \cdots & 1 & 1 & 1 & 1 & 1 & -1 & -1 & \cdots & -1 \end{pmatrix}$$
- where zeroth row has $\frac{n-1}{2}$ ones, for $n = 5(2k+1)$.
- For $5 \leq i \leq n-1$, the i-th row of the matrix $\mathcal{Z}_{5(2k+1)}$ is determined by the following periodicity conditions: $\mathrm{r}_i = T^5(r_{i-5})$.
- For instance, the $\mathrm{r}_6$ is the cyclic shift of $\mathrm{r}_0$ to the right 5 times.

**Example 2.4.**

$$\mathcal{Z}_{5(2k+1)} = \begin{pmatrix}
0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 \\
-1 & 0 & 2 & 2 & 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 \\
-1 & -2 & 0 & 2 & 1 & 1 & 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 & -1 \\
-1 & -2 & -2 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & -1 & -1 & -1 \\
-1 & -1 & -1 & -1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & -1 & -1 & -1 \\
-1 & -1 & -1 & -1 & -1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & -1 & -1 \\
-1 & -1 & -1 & -1 & -1 & -1 & 0 & 2 & 2 & 1 & 1 & 1 & 1 & -1 & -1 \\
-1 & -1 & -1 & -1 & -1 & -1 & -2 & 0 & 2 & 1 & 1 & 1 & 1 & 1 & 1 \\
1 & 1 & -1 & -1 & -1 & -1 & -2 & -2 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \\
1 & 1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & 0 & 1 & 1 & 1 & 1 & 1 \\
1 & 1 & 1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & 0 & 1 & 1 & 1 & 1 \\
1 & 1 & 1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & 0 & 2 & 2 & 1 \\
1 & 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 & -1 & -1 & -2 & 0 & 2 & 1 \\
1 & 1 & 1 & 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 & -2 & -2 & 0 & 1 \\
1 & 1 & 1 & 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & 0
\end{pmatrix}$$

*Note.* All the example above comes from the paper from Prof. Matviichuk.

**Definition 2.9.** We say that the two matrix $B, B'$ are equivalent if there is a permutation matrix $P$ such that $B' = P^{-1}BP$; and projectively equivalent if there is a permutation matrix $P$ and a scalar $\lambda \in \mathbb{C}_{\{0\}}$ such that $B'\lambda P^{-1}BP$.

**Theorem 2.3** (Theorem C)**.** *Let $B$ be a biresidue matrix whose rows sum to zero. Then $B$ has a full smoothable cycle $\iff$ it is projectively equivalent to one of the following matrices:*

- $\mathcal{C}_{n,k,I}$, *where* $1 \leq k < \frac{n}{2}$, *and $I$ is a sequence of zeros and ones of length* $d = \gcd(n, k)$,
- $\mathcal{X}_4, \mathcal{X}_5$,
- $\mathcal{Y}_{2(2k+1)}, k \geq 1$,
- $\mathcal{Z}_{5(2k+1)}, k \geq 1$.

# Chapter 3

# Lean 4 Verification

## 3.1 Construction of Definition

The basic definition is constructed while building the lemma 2.1.

## 3.2 Lemma 2.1

The Lemma 2.1.2 is contributed by my teammate Gao Shiqing Barry, where he first defined the definition of Skew-Symmetry Matrix, which will be used in defining the lemma and build the basis for future work. From now on, without any specification, in this section, all code are used under the importation from Mathlib and also under the namespace SkewMatrixGraph:

```
import Mathlib

namespace SkewMatrixGraph

variable {n : ℕ}
```

### 3.2.1 Skew-Symmetric

Since from the start, we need to build, 1: the Biresidue Matrix, 2: The smoothable undirected edge.

For Biresidue Matrix, note that it is complex matrix, where it is skew-symmetric. Hence

```
def IsSkewSymmetric
    (B : Matrix (Fin n) (Fin n) ℂ): Prop:=
B.transpose = -B
```

Here, we directed construct the matrix B, if it is a Biresidue Matrix, to be Complex valued matrix using the Lean 4 definition, and fix the row and column of B to be a finite value $n$.

Where this construction is not complicated, it is the usual definition which one might visit Mathlib community for similar construction.

### 3.2.2 Smoothable Edge & Lemma 2.1.2

In order to show that an undirected edge is smoothable, the definition must satisfy the 2 conditions: For a biresidue matrix $B$:

1. $b_{ij} \neq 0$.
2. $\forall k \in \{0, 1, ..., n-1\}, k \neq i, j, \ \frac{b_{jk}+b_{ki}}{b_{ij}} \in \mathbb{Z}_{\geq 0}$.

And the definition in the Lean 4 is stated:

```
def IsSmoothableEdge
    (B : Matrix (Fin n) (Fin n) ℂ)
    (i j : Fin n) : Prop :=
i ≠ j ∧
B i j ≠ 0 ∧
∀ k : Fin n, k ≠q i → k≠q j →
    ∃ m : ℕ, B j k + B k i = (m : ℂ) * B i j
```

Note that from above construction, one did not restrict the matrix $B$ to be Biresidue Matrix, the approach is to embed the matrix to biresidue matrix in the hypothesis of the lemma one through the code

```
(B : Matrix (Fin n) (Fin n) ℂ)
(hB : IsSkewSymmetric B)
...
(h_1 : IsSmoothableEdge B i j_1)
(h_2 : IsSmoothableEdge B i j_2)
...
```

Besides, we do casting for the natural number and also use multiplication instead of division, the reasons are as follows:

1. Clears Fractions: It replaces division (/) with multiplication ($*$). This avoids Lean's division-by-zero quirks and allows automated tactics like ring and linear combination to easily solve equations later.
2. Handles Type Casting: The code $(m : \mathbb{C})$ cleanly casts a natural number ($\mathbb{N}$) into a complex number ($\mathbb{C}$) so Lean can perform exact type-safe multiplication.

### 3.2.3 Prove of Lemma 2.1.1

The proof for Lemma 2.1.1 is a standard approach from contradiction, where we use Lean's 'contradiction' or 'False' to set the goal to $\vdash$ False first,

```
  theorem lemma2
(B : Matrix (Fin n) (Fin n) ℂ)
(hB : IsSkewSymmetric B)
(i j_1 j_2 j_3 : Fin n)
(hj12 : j_1 ≠q j_2)
(hj23 : j_2 ≠q j_3)
(hj13 : j_1≠q j_3)
(h_1 : IsSmoothableEdge B i j_1)
(h_2 : IsSmoothableEdge B i j_2)
(h_3 : IsSmoothableEdge B i j_3) :
False :=
```

The proof process lies exactly the same with the mathematical proof, however the part differ from mathematics is its syntactic rigorousness.

For example

1. in standard mathematical proof of Lemma 2.1.1, one usually assumed $B_{ij_3}$ to be non-negative, as this is already defined as a hidden premiss, but Lean requires.

2. The proof of equality of $(q_3 : \mathbb{C}) = ((q_1^* q_2 : \mathbb{Q})\ \mathbb{C})$ after simplification on $(q_3 : \mathbb{C})^* B_{ij_3} = ((q_1^* q_2 : \mathbb{Q})\ \mathbb{C})^* B_{ij_3}$ .

## 3.3 the Families

The construction of Family $\mathcal{Y}, \mathcal{Z}$ are done by my own, while the construction of Family $\mathcal{C}, \mathcal{C}_I$ are contributed by Barry.

### 3.3.1 Family $\mathcal{Y}$

To construct the matrix for Family $\mathcal{Y}$ is not too straightforward. Note by previous chapter, the construction on it requires the cyclic shift operator on the first 2 row: `ROW0, ROW1`.

**Ideal:** While the first idea that comes to mind is to use a recursive definition or an inductive definition of Lean on the matrix; however, since each row are not necessarily constructed under the same magnitude of permutation, I have decided to construct a more general rotation operator for general use, which can also be applied to Family $\mathcal{Z}$.

Before that, the first two row of the matrix are done by constructing the row function, where it maps finite $n$ to a type $\alpha$.

```
def RowVector (n : ℕ) (α : Type) : Type := Fin n → α
```

**Idea:** For the operator $T$, it comes to my mind that directly constructing the permutation of order 2 directly would be too complicated and local, hence, by separating them into small task, then link them together would make the code easier to trace, debug and also modified systematically.

The road-map of partitioning as follows:

Construction of $T$ $\rightarrow$ Usage of $T$ to Row $i$ $\rightarrow$ Construction of $T$ of order $k$

```
def T (k : ℕ) {α : Type} (v : RowVector (2 * (2 * k + 1)) α)
   : RowVector (2 * (2 * k + 1)) α :=
fun i => v (i - 1)
```

↓

```
def useT (k : ℕ) {α : Type} : ℕ → RowVector (2 * (2 * k + 1))
    α → RowVector (2 * (2 * k + 1)) α :=
fun m v  => match m, v with
| 0 ,    v => v
| m+1,  v => T k (useT k m v)
```

↓

```
def Row_Periodic (k : ℕ) : ℕ → RowVector (2 * (2 * k + 1)) ℤ
  | 0    => Row0 k
  | 1    => Row1 k
  | i+2  => useT k 2 (Row_Periodic k i)
```

*Note.* the reason why extra step is added for 'Construction of $T$' $\rightarrow$ 'Usage of $T$ of Row $i$' : First, the `def T` is used to define the function $T$ as a pure logical single step transformation, then `def useT` is used to handle `T` recursively for m times using pattern matching. While lastly for `def Row_Periodic` is used to suit the construction on Family $Y$ where each row from Row2 onwards are constructed by $\mathrm{r}_i = T^2(\mathrm{r}_{i-2})$ where $i \geq 2$.

Note that since the matrix are given in arbitrary size, using natural number makes the construction on perturbation very very hard. Hence, after reading more about the property on `Fin, Finset, N`, it is best to use the modulo property on `Fin.val`, i.e. `Fin n` type is of modulo operation, hence `i+1` := <(`i.val` (mod $n$))+1, `proof`>.

And hence it is direct that the matrix in Family $\mathcal{Y}$ is defined by the following:

```
def Row_Periodic (k : ℕ) : ℕ → RowVector (2 * (2 * k + 1))
    ℤ
| 0     => Row0 k
| 1     => Row1 k
| i+2   => useT k 2 (Row_Periodic k i)

def Y (k : ℕ) : Matrix (2 * (2 * k + 1)) (2 * (2 * k + 1))
    ℤ :=
fun i => Row_Periodic k i.val
```

*Note.* Where here i intended to use $\mathbb{Z}$ instead of $\mathbb{C}$ in the definition is that it is easier to check each construction work by providing examples using the integer version of the matrix (where complex matrix is not able to evaluated by Lean).

*Remark.* Note also that i use AI to help generate a helper function that pretty print the matrix :

By `#eval Y 1`

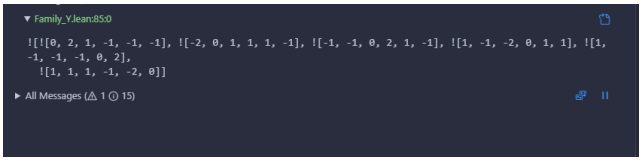



Figure 3.1: picture of `# eval Y 1`

And helper function:

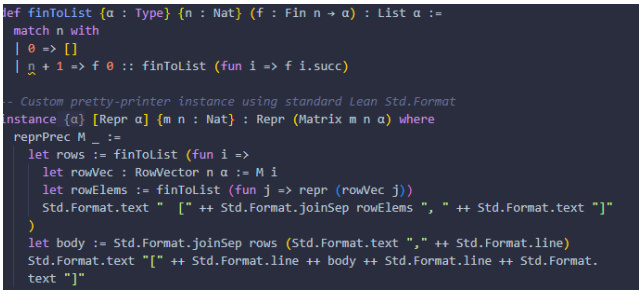



Figure 3.2: picture of helper function

The result `# eval IO.println (repr (Y 1))`

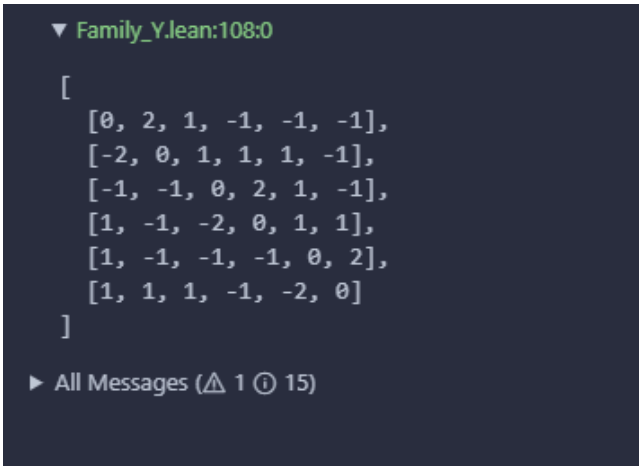



Figure 3.3: `# eval IO.println (repr (Y 1))`

### 3.3.2 Family $\mathcal{Z}$

The Family $\mathcal{Z}$ uses almost exactly the same function and definition, but with the construction `n := (5 * (2 * k + 1))` changed from Family $\mathcal{Y}$.

For example

1.
```
def Row0 (k : ℕ) : RowVector (5 * (2 * k + 1)) ℤ
   := fun i =>
  let n := (5 * (2 * k + 1))
  if i.val == 0 then
    0
  else if i.val < ((n-1) / 2) + 1 then
    1
  else
    -1
```

where other row 1-4 are exactly the same but with the value changed.

2. And the T below:

```
def T (k : ℕ) {α : Type} (v : RowVector (5 * (2 *
   k + 1)) α) : RowVector (5 * (2 * k + 1)) α
```

3. useT:

```
def useT (k : ℕ) {α : Type} : ℕ → RowVector (5 *
   (2 * k + 1)) α → RowVector (5 * (2 * k + 1)) α
```

4. The Row period has 3 extra cases for Family $\mathcal{Z}$

```
def Row_Periodic (k : ℕ) : ℕ → RowVector (5 * (2 *
    k + 1)) ℤ
```

```
| 0     => Row0 k
| 1     => Row1 k
| 2     => Row2 k
| 3     => Row3 k
| 4     => Row4 k
| i+5   => useT k 5 (Row_Periodic k i)
```

5. `#eval IO.println (repr (Z 1))` with use of helper function

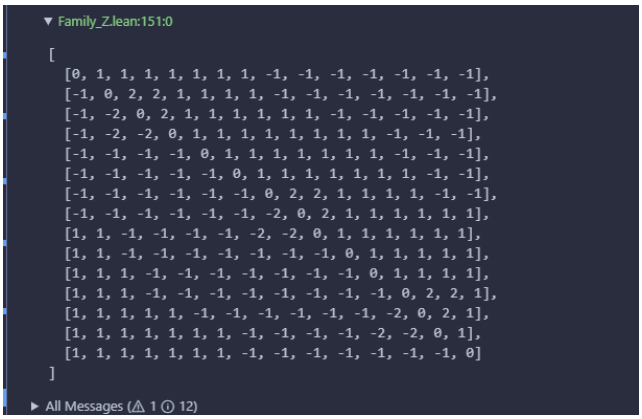



Figure 3.4: picture of `#eval IO.println (repr (Z 1))`

*Remark.* All the construction above are then added, with entries becomes complex-valued $\mathbb{C}$.

## 3.4 Errors and Modifications

Since our Family is constructed without linking the previous construction in Lemma 2.1 with the definition of the Biresidue Matrix and other definition adhered, it would be quite possible that it might cause trouble in future work, to prevent such probability happening, Garry and I decided to use AI to combine our work, which start from using definition of at the Lemma 2.1, to ensure our constructions and works stay consistent.

The updated version of the Families is combined into a single file, where a universal permutation function is made to suit all matrices from Family $\mathcal{C}_I, \mathcal{C}, \mathcal{Y}, \mathcal{Z}$, any matrices defined also satisfy the requirement to be skew-symmetric, biredue (including complex valued), along with integer version (like `def A : Matrix (Fin n)(Fin n) ℤ := ...` ) and listed in the following:

Note that for the definition of Family $\mathcal{Y}, \mathcal{Z}$ is completely identical, except the following:

cyclicRightShift:

```
def cyclicRightShift {n : ℕ} {α : Type*}
    (m : ℕ) (r : Fin n → α) : Fin n → α :=
  fun j => r (((finRotate n).symm^[m]) j)
```

Can be used to replace the

```
def RowVector (n : ℕ) (α : Type) : Type := Fin n → α
```

and that the definition on Row vector for Families changes from

```
def Row1 (k : ℕ) : RowVector (2 * (2 * k + 1)) ℂ := fun i
   => ...
```

to

```
def yRow0 (k : ℕ) : Fin (2 * (2 * k + 1)) → ℂ := fun j  =>
   ...
```

Since the RowVector inputs the Type of `Fin n` to output $\alpha$, which is useless.

Also the new `Families.lean` works in the same namespace as `SkewMatrixGraph` which is the same namespace of in `SkewMatrixGraph.lean` where the definition of biresidue and lemma 1,2 are placed in.

## 3.5 Theorem C

For the Theorem $C$, it is separated into several subsections such that the organization would be improved. The following is the flow of the proof of Theorem $C$:

For each Families, we would first show the sufficiency first, then the necessity.

- The row of entries in each row is zero.
- the Family matrix is Skew-Symmetry.
- for each $k$, the edge $k - (k + 1 \pmod n)$ is colored, where $n$ is the size of the matrix.

The goal is to show that all the matrices come from the 4 Families $\mathcal{C}, \mathcal{Y}, \mathcal{Z}, \mathcal{C}_I$ listed about satisfies the 3 properties.

### 3.5.1 Row sum zero - AddSumLemma.lean

Note that the

### 3.5.2 Skew-Symm -

This part is done by my teammate Garry so we will only briefly go through this section.

### 3.5.3 edge $k - k + 1 \pmod n$ is colored -

Tactics: for each k, the edge k - (k+1 mod n) is colored

Avoids Redundant Index Arithmetic: Working with modular arithmetic over Fin n or ZMod n across general $k$ often creates tedious rewriting steps involving (k + 1 + 1) % n, offset calculations, and cast lemmas. Reducing the claim to a fixed edge (e.g., $0 \to 1$ or $1 \to 2$) dramatically simplifies index manipulations.

## 3.6 Mathematical Verification Process and Frequently Missed Preconditions

Before attempting formalization in an interactive theorem prover, the mathematical definitions must be independently verified across all parameter domains. This process highlights several subtle preconditions that are easily overlooked in informal proofs.

**1. Degenerate Parameter Case ($k = 0$) and the Positivity Constraint ($k > 0$)** A common oversight when working with parametrised vector families over dimension $n = 2(2k + 1)$ is assuming theorems hold for all $k \in \mathbb{N}$. Independent evaluation at $k = 0$ reveals a critical boundary failure:

- When $k = 0$, the dimension collapses to $n = 2(2(0) + 1) = 2$, which implies $n/2 = 1$.
- Evaluating $yRow_0(0, j)$ for $j \in \text{Fin}(2)$:

$$yRow_0(0, 0) = 0 \quad (\text{branch } j = 0), \tag{3.1}$$
$$yRow_0(0, 1) = 2 \quad (\text{branch } j = 1 \text{ takes precedence over } j < n/2). \tag{3.2}$$

- Summing over $j \in \text{Fin}(2)$:

$$\sum_{j=0}^{1} yRow_0(0, j) = 0 + 2 = 2 \neq 0. \tag{3.3}$$

Because index $j = 1$ hits the explicit head condition $j = 1$ before reaching the tail branch, the row sum fails to vanish for $k = 0$. Consequently, the explicit hypothesis $k > 0$ ($k \geq 1$) is strictly required for all row-sum theorems in Family Y.

**2. Independent Interval Verification for Base Row $yRow_0$** For $k \geq 1$, $n = 4k + 2 \geq 6$ and $n/2 = 2k + 1 \geq 3$. Partitioning the domain $[0, n)$ into three non-overlapping regions verifies the net sum independently:

1. **Head Interval** $[0, 2)$: Contains $j = 0$ (value 0) and $j = 1$ (value 2).

$$S_{\text{head}} = 0 + 2 = 2. \tag{3.4}$$

2. **Middle Interval** $[2, n/2)$: Contains $(n/2 - 2) = 2k - 1$ entries, each equal to $+1$.

$$S_{\text{mid}} = (2k - 1) \times 1 = 2k - 1. \tag{3.5}$$

3. **Tail Interval** $[n/2, n)$: Contains $(n - n/2) = 2k + 1$ entries, each equal to $-1$.

$$S_{\text{tail}} = (2k + 1) \times (-1) = -(2k + 1). \tag{3.6}$$

Summing all three regions yields:

$$S_{\text{total}} = S_{\text{head}} + S_{\text{mid}} + S_{\text{tail}} = 2 + (2k - 1) - (2k + 1) = 0. \tag{3.7}$$

**3. Difference Verification for Row** $yRow_1$ Rather than re-evaluating $yRow_1$ over the entire domain, $yRow_1$ is verified as a sparse perturbation of $yRow_0$:

$$\delta Y(k, j) = yRow_1(k, j) - yRow_0(k, j). \tag{3.8}$$

Evaluating point-wise differences across the domain:

- $j = 0$: $yRow_1(0) - yRow_0(0) = -2 - 0 = -2$.
- $j = 1$: $yRow_1(1) - yRow_0(1) = 0 - 2 = -2$.
- $j \in [2, n/2)$: $yRow_1(j) - yRow_0(j) = 1 - 1 = 0$.
- $j \in [n/2, n/2 + 2)$: $yRow_1(j) - yRow_0(j) = 1 - (-1) = +2$ (support size 2).
- $j \in [n/2 + 2, n)$: $yRow_1(j) - yRow_0(j) = -1 - (-1) = 0$.

Net perturbation sum:

$$\sum_{j=0}^{n-1} \delta Y(k, j) = (-2) + (-2) + 0 + (2 \times 2) + 0 = -4 + 4 = 0. \tag{3.9}$$

Thus, $\sum yRow_1 = \sum yRow_0 + \sum \delta Y = 0 + 0 = 0$.

# Chapter 4

# Formalization Insights & Technical Challenges

## 4.1 Finset.sum vs Fin sum

It becomes messy because Lean's internal representation of a finite sum (Finset.sum) is purely structural and recursive, not algebraic.When you look at $\sum_{i=0}^{3} a_i x^i$ on paper, your brain instantly sees a 4-term polynomial. But to Lean:Summation is an abstract operator over a list/finset. Lean doesn't automatically know that $\sum_{i=0}^{N} f(i)$ for $N \leq 3$ is equal to $\sum_{i=0}^{3} f(i)$ without an explicit subset or extensionality proof (Finset.sum-subset or Finset.sum-congr).Bounds checking is strict. In Lean, Finset.range n is a set of natural numbers $\{0, 1, \ldots, n-1\}$. Moving from range (natDegree p + 1) to range 4 requires proving that all "extra" terms between natDegree p + 1 and 4 are strictly zero. Lean will not take $0 * x^i = 0$ for granted inside a sum without unrolling it first.sum-range-succ unrolls from the right. Every time you call rw [sum-range-succ], Lean expands $\sum_{i=0}^{k} f(i)$ into $(\sum_{i=0}^{k-1} f(i)) + f(k)$. Unrolling 4 terms means repeatedly peeling off $f(3)$, then $f(2)$, then $f(1)$, and finally $f(0)$, leaving a chain of additions that you then have to flatten with ring or simp. **Hence we approach with another way.**

## 4.2 Family $Y$

Translating the verified mathematical steps for Family Y into machine-checked Lean 4 code introduces several formalization obstacles related to dependent subtyping, parametric sets, algebraic coercions, and tactic execution order. Below are the core technical challenges alongside structural insights for formalizing piecewise vector families in interactive theorem provers.

**1. Subtyping Obstructions (`Fin n` vs. Plain Natural Indices)** Summing directly over the subtype `Fin n` embeds dependent range proofs $\langle j, h_j \rangle$ into the summation term. This prevents general decision procedures like `omega` from evaluating

inequalities over index variables inside `Finset.sum`.

- **Challenge:** Direct invocation of `Fin.sum_univ_eq_sum_range` fails with type mismatch errors when target functions expect `Fin n`.
- **Solution:** Construct mirror definitions (`yRow0_nat`, `deltaY_nat`) operating directly on $\mathbb{N}$. Use the `change` tactic to align the underlying types before applying range conversion:

$$\sum_{j:\mathrm{Fin}\ n} v(j) \xrightarrow{\texttt{change}} \sum_{j:\mathrm{Fin}\ n} v_{\mathrm{nat}}(j.\mathrm{val}) \xrightarrow{\texttt{Fin.sum_univ_eq_sum_range}} \sum_{j\in\mathrm{range}(n)} v_{\mathrm{nat}}(j). \quad (4.1)$$

- **Insight:** Interactive theorem provers strictly separate dependent types (`Fin n`) from raw scalar types ($\mathbb{N}$). Rather than forcing inline coercions across complex Mathlib summation lemmas, treating domain translation as an explicit two-stage pipeline—defining a unconstrained $\mathbb{N}$-level helper function followed by a type-level alignment via `change`—drastically simplifies downstream tactic automation.

**2. Parametric Set Membership Failures vs. Contiguous Interval Partitioning** Attempting to evaluate finite support sets (e.g., $s = \{0, 1, n/2, n/2 + 1\}$) using discrete set insertion (`Finset.sum_insert`) fails computationally during elaboration because $n/2$ contains the uninstantiated variable $k$.

- **Challenge:** Elaboration tactics like `decide` or `simp` cannot evaluate parametric membership claims like $0 \in \{1, n/2, n/2{+}1\}$ symbolically when variables prevent computational evaluation.
- **Solution:** Replace discrete sets with contiguous interval partitioning (`Finset.Ico`) using `Finset.sum_range_add_sum_Ico`:

$$[0, n) = [0, 2) \cup [2, 2k + 1) \cup [2k + 1, 2k + 3) \cup [2k + 3, n). \quad (4.2)$$

- **Insight:** Computational decision procedures (`decide`) require fully ground terms to unroll finite sets. When symbolic parameters ($k$) are present, declarative interval bounds (`Finset.Ico`) transform finite set-membership evaluation into pure linear integer inequalities, enabling decision procedures like `omega` to resolve boundary conditions symbolically.

**3. Managing Truncated Natural Subtraction and Coercions to $\mathbb{C}$** Evaluating interval cardinalities introduces terms like $\mathrm{card}(\mathrm{Ico}(2, n/2)) = n/2 - 2$. In Lean

4, natural subtraction is truncated ($a - b = 0$ for $a < b$), so casting this subtraction directly to $\mathbb{C}$ yields `((n / 2 - 2 : ℕ) : ℂ)`.

- **Challenge:** Invoking `push_cast` directly fails or creates ill-formed target expressions because truncated natural subtraction does not distribute linearly over field coercions without an explicit proof that $2 \leq n/2$.
- **Solution:** Resolve natural subtractions entirely within $\mathbb{N}$ first (e.g., `have h1 : n / 2 - 2 = 2 * k - 1 := by omega`), rewrite the simplified natural expression into the goal, apply `Nat.cast_sub`, and only then invoke `push_cast` and `ring`.
- **Insight:** Field tactics (`ring`) operate under standard field axioms and cannot reason about non-algebraic truncated operators. Completing all arithmetic reductions in $\mathbb{N}$ prior to casting into a characteristic-zero field ($\mathbb{C}$) guarantees that field tactics operate on true ring invariants.

**4. Strict Tactic Ordering for Branch Elimination** When evaluating conditional statements (`split_ifs`) over definitions containing `let n := 2*(2k+1)`, `omega` cannot eliminate false conditional branches (e.g., $j < n/2$ when $j \geq n/2$) unless $n/2$ is explicitly unfolded. Furthermore, calling `ring` too early can leave open goals on false algebraic equalities (e.g., $-1 = 3$).

- **Challenge:** Blindly applying ring simplification to conditional goals can simplify contradictory branches into unprovable field equations.
- **Solution:** Unfold $n$ and supply the explicit division identity `have hn_div : n / 2 = 2 * k + 1 := by omega` prior to splitting conditionals. Enforce a strict tactic pipeline:

```
split_ifs <;> first | rfl | omega | ring
```
(4.3)

- **Insight:** Equational tactics like `ring` are logically blind to surrounding context hypotheses. Structuring proof combinators into a strict power hierarchy—definitional reduction (`rfl`) → integer bound contradiction (`omega`) → ring simplification (`ring`)—prevents the prover from getting trapped in unreachable or logically contradictory goal states.

While Family Y is structurally simpler than Family Z (consisting of only two seed rows over dimension $n = 2(2k + 1)$), formalizing its row sum properties in Lean 4 reveals several subtle interaction issues between interactive theorem proving tactics, type coercions, and decision procedures.

Table 4.1: Formalization Challenges, Observed Tactics, and Structural Insights

| Challenge Area | Observed Tactic Failure | Core Structural Insight |
|---|---|---|
| **Subtyping Barrier** | Type mismatch on `Fin.sum_univ_eq_sum_range` | Separate `Fin n` from $\mathbb{N}$ definitions; align via `change`. |
| **Parametric Support** | `decide` / dependent elimination failure on set insertion | Replace discrete set insertions with contiguous `Finset.Ico` range splits. |
| **Truncated Casts** | Pattern $\uparrow (n/2)$ not found during `rewrite` | Perform natural subtraction reductions in $\mathbb{N}$ before field coercion. |
| **Branching Pipeline** | Unsolved false goals (e.g., `-1 = 3`) after `ring` | Enforce execution order: `rfl` $\rightarrow$ `omega` $\rightarrow$ `ring` to filter false branches. |

Below is an analysis of the primary technical difficulties encountered and the key design guidelines established during formalization.

**1. Essential Mathematical Condition ($k > 0$)** A critical prerequisite discovered during proof construction is the strict positivity requirement $k > 0$ ($k \geq 1$). For the degenerate case $k = 0$, the dimension collapses to $n = 2(2(0) + 1) = 2$, yielding $n/2 = 1$. In this scenario, index $j = 1$ triggers the explicit head branch `if j = 1 then 2` instead of entering the tail range, resulting in:

$$\sum_{j=0}^{1} yRow_0(0, j) = yRow_0(0, 0) + yRow_0(0, 1) = 0 + 2 = 2 \neq 0. \tag{4.4}$$

Thus, attempting to prove the theorem for unconstrained $k \in \mathbb{N}$ is mathematically impossible; $k > 0$ must be explicitly passed as an explicit hypothesis (`hk : 0 < k`).

**2. Bridging `Fin n` Subtyping to Plain Natural Indices** Summing directly over the subtype `Fin n` forces Lean to carry dependent bounds $\langle j, h_j \rangle$ inside sum expressions. This hampers integer decision procedures like `omega`, which cannot easily unpack structured subtypes inside `Finset.sum`.

- **Pitfall:** Applying `Fin.sum_univ_eq_sum_range` directly fails with a type mismatch error if the target expression is defined on `Fin n`.
- **Remedy:** Define plain natural-number representations (`yRow0_nat`, `deltaY_nat`) operating on $\mathbb{N}$. Align types using the `change` tactic before invoking `Fin.sum_univ_eq_sum_ran`

$$\sum_{j:\mathrm{Fin}\ n} v(j) \xrightarrow{\texttt{change}} \sum_{j:\mathrm{Fin}\ n} v_{\mathrm{nat}}(j.\mathrm{val}) \xrightarrow{\texttt{Fin.sum_univ_eq_sum_range}} \sum_{j \in \mathrm{range}(n)} v_{\mathrm{nat}}(j). \tag{4.5}$$

**3. Contiguous Interval Decomposition vs. Parametric Set Membership** Attempts to isolate support indices using discrete finite sets (e.g., $s = \{0, 1, n/2, n/2+1\}$) and `Finset.sum_insert` fail in Lean 4 because $n/2$ contains the free variable $k$. Computation tactics like `decide` or `simp` cannot evaluate parametric set membership $0 \in s$ at elaboration time.

- **Remedy:** Partition the entire range $[0, n)$ into contiguous closed-open intervals (`Finset.Ico`) using `Finset.sum_range_add_sum_Ico`:

$$[0, n) = [0, 2) \cup [2, 2k+1) \cup [2k+1, 2k+3) \cup [2k+3, n). \tag{4.6}$$

  This enables pure arithmetic bound checks via `omega` across every sub-interval.

**4. Managing Natural Subtraction and Coercions to $\mathbb{C}$** When evaluating interval cardinality sums over $\mathbb{C}$, terms like $\text{card}(\text{Ico}(2, n/2)) = n/2 - 2$ produce complex-coerced natural subtractions: `((n / 2 - 2 : ℕ) : ℂ)`. Calling `push_cast` directly fails or creates ill-formed target expressions because natural subtraction is non-commutative.

- **Remedy:** Always evaluate natural subtraction identities at the $\mathbb{N}$ level first (e.g., `have h1 : n / 2 - 2 = 2 * k - 1 := by omega`), rewrite them into the goal, apply `Nat.cast_sub`, and only then call `push_cast` followed by `ring`.

**5. Strict Tactic Ordering for Branch Elimination** When evaluating conditional branching (`split_ifs`) over definitions containing `let n := 2*(2k+1)`, `omega` cannot discharge false branches (e.g., $j < n/2$ when $j \geq n/2$) unless $n/2$ is explicitly expanded. Furthermore, the `ring` tactic does not fail on false equalities (like $1 = -1$), leaving open goals if called too early.

- **Remedy:** Pre-evaluate $n/2 = 2k+1$ via `dsimp [n]; omega` and rewrite it into the context before calling `split_ifs`. Always structure tactic combinators as:

```
split_ifs <;> first | rfl | omega | ring
```
(4.7)

  This ensures `omega` eliminates arithmetic contradictions before `ring` attempts complex ring simplification.

## 4.3 zRowk_sum_zero

A standard technique for establishing that a row vector $v \in \mathbb{C}^N$ satisfies $\sum_{j=0}^{N-1} v_j = 0$ relies on point-wise anti-symmetry with respect to the negation operator in the cyclic

Table 4.2: Key Reminders for Lean 4 Sequence Sum Formalization

| **Issue Area** | **Common Tactic Error** | **Correct Strategy / Solution** |
|---|---|---|
| **Type Conversion** | `rewrite failed: pattern` $\uparrow$ $(n/2)$ `not found` | Prove $n/2 - c = f(k)$ in $\mathbb{N}$, `rw`, then `Nat.cast_sub` & `push_cast`. |
| **Set Support** | `decide` / dependent elimination failure on set insertion | Avoid `sum_insert`; use contiguous `Finset.Ico` interval splitting. |
| **Branching** | Open goals like `-1 = 3` after `ring` | Use `first \| rfl \| omega \| ring` so `omega` clears impossible branches first. |
| **Natural Division** | `omega` fails on $n/2$ inside `let` bindings | Unfold $n$ and supply `have : n / 2 = 2*k + 1 := by omega` explicitly. |

group $\mathbb{Z}/N\mathbb{Z}$:

$$v(-j) = -v(j) \quad \forall j \in \mathrm{Fin}(N). \tag{4.8}$$

When Eq. (1) holds, pairing indices $j$ and $-j$ yields $2\sum_j v_j = \sum_j (v_j + v_{-j}) = 0$, implying $\sum_j v_j = 0$. While this property holds directly for the base seed vector $zRow_0(k)$, it fails for the remaining seed vectors $zRow_i(k)$ $(i \in \{1, 2, 3, 4\})$.

**Failure of Anti-Symmetry** Consider $zRow_1(k)$ defined over dimension $N = 5(2k+1) \geq 5$. At $j = 0$, $zRow_1(k, 0) = -1$. Because $-0 \equiv 0 \pmod N$, we have:

$$zRow_1(k, 0) + zRow_1(k, -0) = -1 + (-1) = -2 \neq 0. \tag{4.9}$$

Similarly, for $j = 1$, $zRow_1(k, 1) = 0$, whereas $-1 \equiv N - 1 \pmod N$ falls into the tail branch $(N - 1 \geq \frac{N-1}{2} + 1)$, giving $zRow_1(k, N - 1) = -1$. Thus:

$$zRow_1(k, 1) + zRow_1(k, -1) = 0 + (-1) = -1 \neq 0. \tag{4.10}$$

Consequently, the anti-symmetry proof cannot be applied to rows $i \geq 1$.

**Perturbation Decomposition Strategy** To circumvent this obstruction, we decompose each non-antisymmetric seed row $zRow_i(k)$ into the sum of the antisymmetric base row $zRow_0(k)$ and a sparse correction profile $\delta_i(k)$:

$$zRow_i(k, j) = zRow_0(k, j) + \delta_i(k, j). \tag{4.11}$$

By linearity of summation, the vanishing of the total row sum reduces to proving that the support of $\delta_i(k)$ sums to zero:

$$\sum_{j\in\mathrm{Fin}(N)} zRow_i(k,j) = \underbrace{\sum_{j\in\mathrm{Fin}(N)} zRow_0(k,j)}_{=0 \text{ (by anti-symmetry)}} + \sum_{j\in\mathrm{Fin}(N)} \delta_i(k,j). \tag{4.12}$$

Defining $M = \frac{N-1}{2}$, the correction profiles $\delta_i(k,j) = zRow_i(k,j) - zRow_0(k,j)$ have small finite support independent of $N$:

- **Row 1:** $\delta_1(k,0) = -1$, $\delta_1(k,1) = -1$, $\delta_1(k,2) = +1$, $\delta_1(k,3) = +1$.

$$\sum_j \delta_1(k,j) = (-1) + (-1) + 1 + 1 = 0. \tag{4.13}$$

- **Row 2:** $\delta_2(k,j)$ receives head adjustments at $j \in \{0,1,2,3\}$ summing to $-4$, and a cutoff shift of $+2$ at $j \in \{M+1, M+2\}$.

$$\sum_j \delta_2(k,j) = (-1-3-1+1) + 2(2) = -4 + 4 = 0. \tag{4.14}$$

- **Row 3:** Head adjustments sum to $-8$, balanced by a cutoff shift of $+2$ across $j \in \{M+1, M+2, M+3, M+4\}$.

$$\sum_j \delta_3(k,j) = (-1-3-3-1) + 2(4) = -8 + 8 = 0. \tag{4.15}$$

- **Row 4:** Head adjustments sum to $-8$ across $j \in \{0,1,2,3,4\}$, balanced by the same $+8$ cutoff shift.

$$\sum_j \delta_4(k,j) = (-1-2-2-2-1) + 2(4) = -8 + 8 = 0. \tag{4.16}$$

This establishes $\sum_j zRow_i(k,j) = 0$ for all $i \in \{0,\ldots,4\}$.

# Chapter 5

# Conclusion & Future Work